2022 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM) | 979-8-3503-9421-4/22/$31.00 ©2022 IEEE | DOI: 10.1109/CCEM57073.2022.00012

Optimising resource allocation in a power and cost-aware multi-cloud environment using Multi-Objective Antlion Optimisation algorithm

1st Prithvi Anil Kumar
Dept of Computer Science and Engineering, PES University
Bengaluru, India
prithvianilk@gmail.com

2nd Spandan Sar
Dept of Computer Science and Engineering, PES University
Bengaluru, India
spandan.sar@gmail.com

3rd Surya Raj
Dept of Computer Science and Engineering, PES University
Bengaluru, India
surya.pesu@gmail.com

4th Varun MK
Dept of Computer Science and Engineering, PES University
Bengaluru, India
mkvarun2001@gmail.com

5th HL Phalachandra
Formerly Dept of Computer Science and Engineering, PES University
Bengaluru, India
hl.phala@gmail.com

6th Prafullata K Auradkar
Dept of Computer Science and Engineering, PES University
Bengaluru, India
prafullatak@pes.edu

***Abstract*—With the proliferation of cloud computing and the necessity to work across multiple clouds, there is a strong need to schedule tasks to cloud resources effectively by factoring in the constraints of makespan, cost, and power utilisation.**

In this paper, a meta-heuristic optimisation algorithm (Quasi-oppositional Multi-objective Antlion Optimizer based on Differential Evolution) is utilised to allocate a set of cloud tasks to heterogeneous containers and simultaneously allocate these containers to virtual machines spanning across multiple Cloud Service Providers. Experiments conducted using the CloudSim framework on two cloud task workloads show that the MOALO algorithm performs competitively in terms of makespan and significantly better in cost and power utilisation when compared to scheduling algorithms like Min-Min and Max-Min.

***Keywords*—Multi-cloud, Multi-objective optimisation, Containers, CloudSim, Multi-objective Antlion Optimisation, Meta-heuristic algorithm**

I. Introduction

In the modern software environment, applications, services, and projects are built using cloud computing technologies, allowing developers to use computing, storage, and networking infrastructure and platforms in an on-demand, pay-as-you-go fashion. Multi-cloud is a cloud computing deployment model that involves a combination of multiple private or public Cloud Service Providers (CSPs) for various cloud services [1]. This provides a lot of advantages over a single cloud like the ability to choose the best set of services from each CSP, higher availability, reduced cost, better disaster recovery, etc. This not only helps in performance and scalability but also prevents vendor lock-in [2].

In a multi-cloud environment, efficient container allocation is important, as most modern-day applications utilise containers for deployments [2]. Containers are used by CSPs to provide elasticity to their users. As a result, elastic containerised platforms are being increasingly used across multiple clouds.

Scheduling can be defined as the allocation of resources and tasks over a given time frame along with the goal of working with one or more constraints. The allocation or scheduling problem, being an NP-complete problem, implies that it is infeasible to find an optimal solution in a realistic time frame. Hence, there exist multiple algorithms that present an approximate solution in polynomial time [3]. Within the cloud computing domain, task scheduling refers to the allocation of cloud tasks to cloud resources, such as containers and VMs.

Optimized scheduling of cloud tasks is required to obtain the maximum performance out of the available resources. One way to measure performance is to minimise total cloud task makespan. In this context makespan is defined as the total time taken to complete execution of all cloud tasks. Since, running cloud resources like VMs, incur a certain cost, reducing the overall monetary cost of a cloud resource configuration is required while allocating resources. Finally, since different machines may consume different amounts of power given a certain load, running tasks on a machine that consumes excessive power will be detrimental to the overall carbon footprint of a datacenter. A scheduling algorithm must be performant, cost-effective and power-aware to meet the QoS requirement of the end users.

The main contributions of this paper are the following,

1) A novel approach of modeling a containerised, cost and power-aware multi-cloud environment in CloudSim.
2) Optimisation of task and resource allocations to minimise total execution time, monetary cost, and power utilisation metrics.

979-8-3503-9421-4/22/$31.00 ©2022 IEEE
DOI 10.1109/CCEM57073.2022.00012

II. Related Work

There are many types of task scheduling algorithms that assign tasks to containers, containers to various VMs (Virtual machines), and VMs to hosts. A review of work done in cloud scheduling algorithms gives a great insight into how tasks are assigned to various VMs [4]. These algorithms reduce makespan, average waiting time, power utilisation, etc. There are various deterministic algorithms like Shortest Job First [5], First Come First Serve [6] and Min-Min [7] which aim to optimise a single constraint like makespan.

Past research involving cloud scheduling algorithms indicates that meta-heuristic algorithms provide better results than deterministic scheduling algorithms while optimizing single or multiple objective functions alike [8] [9] [10]. Machine learning-based algorithms [11], Particle Swarm Optimisation [12] and similar Genetic Algorithms [13] which work towards the optimisation of various objectives are popular techniques used in this field. A few examples are mentioned below. The Moth Search Algorithm with Differential Evolution mimics moths in nature by looking at how they move towards a light source. This algorithm increases throughput and utilises differential evolution to improve its exploitation ability [14]. The Whale Optimisation Algorithm is a bio-inspired, optimisation algorithm that mimics how whales hunt their prey with a spiral behaviour [15]. The Lion Optimisation Algorithm is a multi-objective optimiser that mimics how lions hunt in the wild [16]. The Multi-objective Antlion Optimisation Algorithm (MOALO) is based on how antlions prey on ants in nature [17]. This algorithm is shown to work well on optimising multiple objective functions compared to other such bio-inspired techniques. Past research aims to minimise makespan by utilising the MOALO algorithm to optimise task allocation. It is shown that MOALO has a better improvement factor and faster convergence rate in a large search space compared to other well-known algorithms [18]. This makes MOALO well-suited for multi-objective scheduling problems in the cloud computing domain.

There also has been some past work in designing algorithms that aim to optimise container orchestration across multiple clouds [19].

III. Theoretical Framework

A. Definitions and Equations

This work models a multi-cloud environment using various cloud resources.

1) CSP: A Cloud Service Provider is modelled as a set of datacenters, where each datacenter is modelled as a set of physical servers.

2) Host: A physical server (host) is modelled as an entity with a set of VMs running on it, total MIPS (Million Instructions Per Second) per Processing Element (PE), and number of PEs. Each PE is a physical computing core. Along with this, each Host type has a power utilisation specification, which is a list of size 11, containing power utilisation of a host for each unit of utilised host MIPS at an interval of 10%, starting from 0% till 100%.

$$host_i = \{\ host\_id,\ vm\_list,\ mips,\ number\ of\ PEs,\ power\ model\ spec\ \} \tag{1}$$

3) Virtual Machine: A VM is a virtualised entity that is provisioned within a Host. Each VM is modelled to host multiple containers, total MIPS per PE and number of PEs. It also contains the total execution time ($exectime$) after starting the VM and the cost (per sec) of running the VM.

$$vm_i = \{\ vm\_id,\ container\_list,\ mips,\ number\ of\ PEs,\ exectime,\ cost\ \} \tag{2}$$

4) Container: A container is a virtualised environment within a VM. Since it is simply a lighter layer of virtualisation, it is modelled similar to a VM.

$$container_i = \{\ vm\_id,\ mips,\ number\ of\ PEs\ \} \tag{3}$$

5) Cloudlet: A cloudlet is modelled as a generic cloud task. Each cloudlet can be represented by its length. This represents the number of compute instructions associated with a task.

$$cloudlet_i = \{\ cloudlet\_id,\ length\ \} \tag{4}$$

6) Container execution time: The execution time of a container is defined using the following formula

$$container_i.exectime = \frac{\sum cloudlet_j.length}{(container_i.mips \times container_i.pes)}$$
$$\forall\ 1 \leq j \leq number\ of\ cloudlets\ allocated\ to\ container_i \tag{5}$$

7) VM execution time: The execution time of a VM is defined using the following formula

$$vm_i.exectime = max(container_j.exectime)$$
$$\forall\ 1 \leq j \leq\ number\ of\ containers\ allocated\ to\ vm_i \tag{6}$$

8) Fitness Functions: An allocation's fitness is determined by 3 absolute values :

- Makespan of the entire system.
- Monetary Cost of running cloud resources.
- Total Power utilisation of running physical resources like Hosts.

9) Makespan: Makespan of the entire multi-cloud system refers to the total difference between start time and end time of all cloudlets after execution. Since VMs are the virtual resources that eventually execute these tasks, a VMs execution time is used to calculate the makespan of the tasks allocated to that VM. Given K VMs,

$$makespan = max(vm_i.exectime)\ \forall\ 1 \leq i \leq K \tag{7}$$

10) Cost: Acquiring and running VMs on the cloud costs a certain amount of monetary value per second. Each CSP has differing cost factors for a second of running a similar type of VM. The total cost of a VM configuration on the cloud can be represented using the following formula.

$$cost = \sum_{i=1} vm_i.exectime \times vm_i.cost \tag{8}$$

11) Power Utilisation: Keeping physical resources up and running utilises a certain power within a data center. For a single host, this depends on the utilisation of the host with respect to its MIPS.

$$host_i.power = \sum_j host_i.utilisation(j) \times (container_j.exectime - container_{j-1}.exectime) \quad \forall\ 1 \leq j \leq number\ of\ containers\ in\ host_i \tag{9}$$

And, $host_i.utilisation(x)$ is defined as

$$host_i.utilisation(x) = \frac{mips\ utilised\ with\ x-1\ containers\ completed}{host_i.mips} \tag{10}$$

12) Allocation matrices: An allocation is represented by two matrices.

Given,

$N =$ number of cloudlets

$M =$ number of containers

$K =$ number of VMs

The first matrix, cloudletMap is an $N \times M$ matrix, consisting of 0s and 1s, where,

$$cloudletMap[i][j] = 0 \quad if\ cloudlet_i\ is\ not\ mapped\ to\ container_j$$
$$cloudletMap[i][j] = 1 \quad if\ cloudlet_i\ is\ mapped\ to\ container_j \tag{11}$$

Similarly, the second matrix, containerMap is an $M \times K$ matrix, consisting of 0s and 1s, where,

$$containerMap[i][j] = 0 \quad if\ container_i\ is\ not\ mapped\ to\ vm_j$$
$$containerMap[i][j] = 1 \quad if\ container_i\ is\ mapped\ to\ vm_j \tag{12}$$

B. Multi-Objective Antlion Optimisation Algorithm

This research looks into utilising a nature-inspired multi-objective optimisation algorithm called Multi-Objective Antlion Optimisation (MOALO) to optimise cloud resource allocations in a multi-cloud environment [20]. This algorithm mimics the hunting patterns of antlions wherein ants develop random walks and antlions create pits to trap ants as prey.

The MOALO algorithm has been used to solve many real-world engineering problems like designing a brushless DC wheel motor, disk brake, cantilever beam etc.

MOALO is an improvement over the Ant Lion Optimizer algorithm [17], and is aimed at solving multi-objective optimisation problems. In comparison with other bio-inspired algorithms like MOPSO and NSGA-II, MOALO has a better convergence rate and better coverage in the search space, which helps in finding the best Pareto-optimal front [20].

Given an NP-complete problem and multiple objective functions to minimise, MOALO generates a set of Pareto-optimal (non-dominated) solutions. A Pareto-optimal solution is one in which a value of an objective function cannot be improved without degrading the values of the other objective functions. From the set of Pareto-optimal solutions, a feasible solution is identified based on the problem-specific requirements. Fig. 1 shows a workflow of the MOALO algorithm.

IV. Design and Modelling

A. Implementation of MOALO for cloud resource allocation

In the first step, the multi-cloud environment is set up. This includes configuration of CSPs that house various hosts, VMs, and containers.

The MOALO algorithm is used to optimise the allocation of various cloud resources, given three objective functions as constraints. These are total makespan, total CSP cost and power utilised to run all cloud tasks (cloudlets) in the multi-cloud simulation.

Through the course of this research, it is observed that MOALO has a tendency to fall into the local optimum and does not converge at the global optimum. Hence, an improved Quasi-oppositional multi-objective antlion optimisation algorithm based on differential evolution is utilised [21].

The differential evolution strategy helps in improving an ant's random walk mechanism by utilising the positions of the elite antlion, randomly selected antlion, and the ant's positions.

Quasi-opposition learning strategy helps in developing a diverse ant population, wherein the ant population is generated by choosing the ants with the best fitness from the original and the quasi-opposite ant populations. This is based on the proof that quasi-opposite points have a higher chance to be closer to the optimal solution [22].

The combination of both these strategies gives MOALO an increased search space exploration and global optimum convergence capabilities.

1) Algorithm: The following section gives an overview of the improved MOALO algorithm.

The initial position of every ant is randomly generated and the diversity of the population is improved by combining this population with the quasi-opposite ant populations.

$$Ant_i = \{\ cloudletMap, containerMap\ \} \tag{13}$$

Each ant is defined as given in Eq. 13. Where the $i'th$ ant contains a set of its own mapping between all cloudlets to containers and mapping between all containers to VMs. These maps are represented as matrices where each cell is bounded in the interval [0, 1].

Each ant presents a candidate solution to the optimisation problem.

The Quasi-oppositional based learning technique is used to generate a diverse initial random population in an optimal way. Using this, a random ant x and its opposite ant position is calculated.

$$ox = a + b - x \tag{14}$$

This ant's quasi-opposite ant is calculated using the following equation:

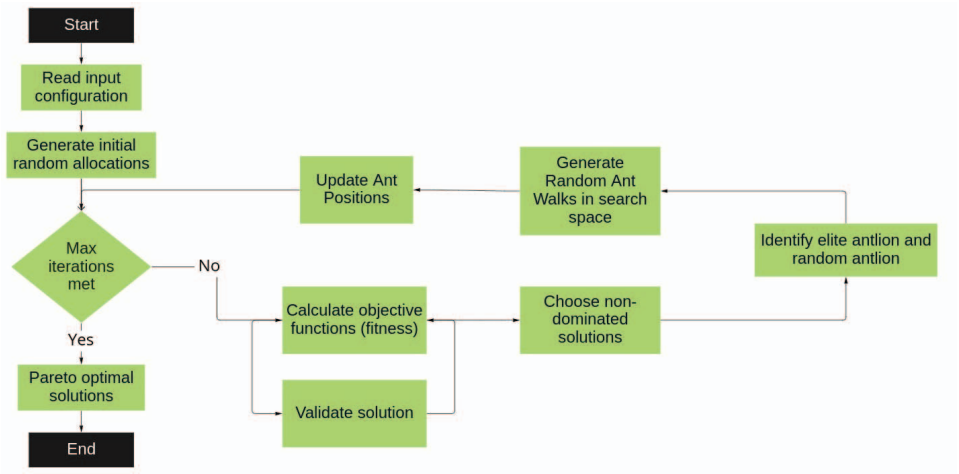

Fig. 1. MOALO Algorithm

Algorithm 1: Improved MOALO algorithm

Result: *Set of non − dominated ant positions and fitness values (Pareto − optimal solutions for defined objective functions)*

1: *Define objective functions.*
2: *Initialise ant positions bounded within search space using quasi − opposition learning.*
3: $N \leftarrow Number\ of\ Iterations$
4 : **while** N $\neq 0$ **do**
5 : *Calculate fitness values of each ant using the defined objective functions.*
6: *Find set of non − dominated ant positions.*
7: *Reduce number of ants in search space based on fitness for faster convergence.*
8: *Select an elite antlion using Roulette wheel selection from archived antlion population.*
9: *Select a random antlion from archived antlion population.*
10: *Random walk of every ant in the search space is updated using the positions of the elite antlion and random antlion chosen.*

$$qox = rand\ (\frac{a+b}{2},\ ox) \tag{15}$$

The set of all ant positions are stored as a vector as given in Eq. 16 where m is the number of ants.

$$Ant\ Positions\ =\ [Ant_1,\ Ant_2, ..., Ant_m] \tag{16}$$

Fitness of an ant depends on its position and is evaluated using objective functions. The fitness of an ant is stored as a vector as given in Eq. 17 where $f(Ant_i) = [f_1(Ant_i), f_2(Ant_i), f_3(Ant_i)]$ and k is the number of objective functions to optimise.

$$Fitness\ of\ ants = [f(Ant_1),\ f(Ant_2), ..., f(Ant_m)] \tag{17}$$

The ants move around the search space to explore better solutions using random walks. The position of ants in these random walks is not only influenced by the elite antlion but also by the entire archived population of antlions, thus introducing local information and context of the antlion while updating.

The ant's position is updated using the following formulae.

$$\begin{aligned} Ant_i = \overline{Antlion_c} &+ F_1 \times (Antlion_{elite}\ -\ Antlion_{rand}) \\ &+ F_2 \times (Antlion_{elite} -\ Ant_i) \\ &+ F_3\ \times\ (Antlion_{rand} -\ Ant_i) \end{aligned} \tag{18}$$

$$\text{Where, } F_i \sim \mathcal{N}(0.5,\ 0.3) \tag{19}$$

$$\begin{aligned} \overline{Antlion_c} = \omega_1 &\times Antlion_{elite} \\ &+ \omega_2 \times Antlion_{rand} + \omega_3 \times Ant_i \end{aligned} \tag{20}$$

$$\sum_i \omega_i = 1,\ \omega_i = \frac{p_i}{\sum_i p_i} \tag{21}$$

$$p_1 = 1,\ p_3 = rand(0.7,\ 1),\ p_2 = rand(0.5,\ p_3) \tag{22}$$

where ω_i in Eq. 21 represents the fraction of contributions from ant_i, $Antion_{elite}$ and $Antlion_{rand}$ towards $\overline{Antlion_c}$ These random walks are influenced by 3 factors,

- The elite antlion is selected using the Roulette wheel.
- A randomly selected antlion from the archived antlion population.
- The current ant's position.

The position of the elite antlion is chosen from one of the non-dominated ant's positions, and its position updates itself every iteration.

On obtaining the set of non-dominated (Pareto-optimal) solutions, the fitness value of each ant is obtained as the

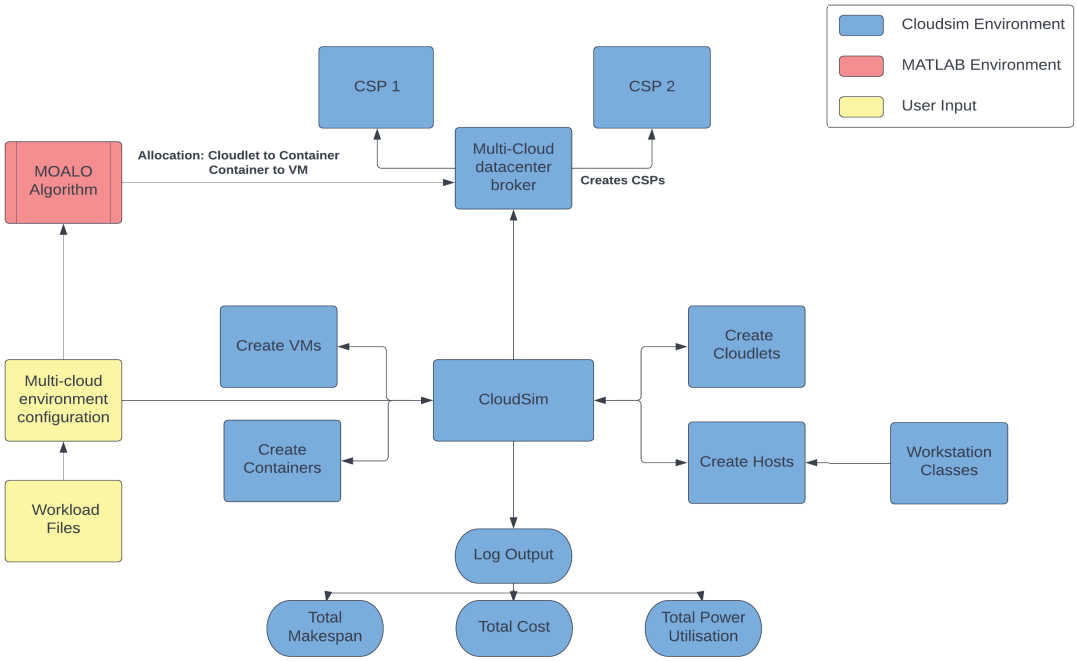

Fig. 2. High Level Design Diagram

Pareto-optimal solution from which a solution with the least variance is chosen.

As the final step, the $containerMap$ and $cloudletMap$ of the chosen solution are converted to an allocation matrix with only 0s and 1s, where a 1 represents a resource being allocated to another resource. A cell value in the matrix is downgraded to a 0 if the value is $<= 0.5$, else it is upgraded to 1.

This allocation matrix is the final optimized allocation, which minimises the defined objective functions.

B. CloudSim Framework

Comparing and evaluating algorithms and implementation of a multi-cloud environment using real-world cloud service providers is challenging due to the lack of infrastructure, integration between the CSPs and the cost to run these servers. In this paper, modelling and simulation is done in CloudSim. CloudSim is a Java framework to simulate a cloud computing environment, execute experiments and test results [23].

CloudSim entities are used to define low-level resources of CSPs, such as Hosts, VMs, and cloudlets. A CSP entity was created which is used to host multiple datacenters with different costs, machines, and configurations.

CloudSim is also extensive enough to provide its users with highly granular statistics like cloud task execution time and power utilisation of a datacenter. ContainerCloudsim [24] is an extension to CloudSim, which adds containers as a key resource in the cloud environment, and subsequently models all other resources, like VMs, Hosts, etc, around the presence of a container.

In a multi-cloud environment, a central datacenter broker interacts with multiple CSPs to allocate resources and tasks. Cloudlets are sent to the broker and allocates them to different CSPs.

Several mappers were implemented to map different entities and schedule tasks. These mappers are integrated with the broker to make sure the scheduling and allocation happen systematically.

C. Implementation of Max-Min and Min-Min algorithms

The MOALO algorithm is compared against Max-Min and Min-Min algorithms which are implemented within the CloudSim framework. The Max-Min algorithm aims to schedule the tasks with a larger size first [25]. A given task is assigned to the container with the least overall completion time, taking into consideration the tasks already assigned to the container. A similar container assignment scheme is used in Min-Min, but, the Min-Min algorithm aims to schedule the tasks with a smaller size first [7].

D. High Level Design of the system

Fig. 2 depicts the high-level design of the multi-cloud environment and the integration of CloudSim with the MOALO algorithm. The multi-cloud setup exists in the CloudSim framework whereas the MOALO algorithm is coded and modeled in MATLAB. A multi-cloud environment configuration file is generated by reading the workload files provided by the user. This configuration is fed to the CloudSim environment as well as to the MOALO algorithm. After training on

the given configuration, the MOALO algorithm produces an optimal solution and writes it to an allocation solution file. Within Cloudsim, the configuration file is used to set up the multi-cloud environment and used to create CSPs, a multi-cloud datacenter broker, datacenters, VMs, and cloudlets. The allocation solutions are sent to the datacenter broker to set up and start the simulation of the system. After successful completion of the simulation, the system generates a report, containing a table of the completed cloudlets, their start and end time and the container allocated to it. Subsequently, the overall makespan, power utilisation per CSP, and cost per CSP is generated.

V. Results

A. Experimental setup

The simulated multi-cloud environment consists of two CSPs, where each CSP has a unique host type with differing power consumption and cost (per second) associated with acquiring and running VMs. The power consumption of each host depends on its CPU utilisation at a particular time.

CSP 1 consists of Host type HPEProLiantDL360Gen10 [26], while CSP 2 consists of Host type LenovoThinkSystemSR650 [27].

System specifications of the simulated host types and CSPs are tabulated in Table 1 and 2.

TABLE I

Host Type	**HPEProLiant**	**LenovoThinkSystem**
Clock rate(GHz)	2.5	2.1
No Of Cores	56	56
RAM(GB)	96	192
VM Cost(per sec)	8 units	5 units

TABLE II

	CSP 1	**CSP 2**
Host Type	HPEProLiant	LenovoThinkSystem
Number of Hosts	1	1
Number of VMs	2	2
VM Cost(per sec)	8 units	5 units

TABLE III

Parameter	**Units**
Number of Containers	8
Container MIPS	10 - 500 MIPS
VM MIPS	2000 MIPS

Table III shows the configurations of various cloud resources used in the simulation. The container MIPS used in the configuration is generated via a random generation whose minimum and maximum ranges are mentioned.

B. Workloads

The MOALO, Max-Min, and Min-Min scheduling algorithms are run against the NASA Ames iPSC/860 [28] workload and KTH IBM SP2 [29] workload under similar cloud configurations. The NASA workload consists of 18,239 cloud tasks over three months, which is a mix of interactive and batch-processing jobs. On the other hand, KTH SP2 workload consists of 28,476 accounting cloud tasks from over 11 months from the Swedish Royal Institute of Technology. Using these workloads the algorithms are tested and results are compared on subsets of 500, 1000, 1500, and 2000 cloud tasks.

TABLE IV
Cost units

NASA Ames iPSC/860			
No of cloudlets	**MOALO**	**Min-Min**	**Max-Min**
500	1,418	2,166	1,857
1000	3,104	4,097	3869
1500	5,985	6,492	6,080
2000	7,546	8,340	7,638
KTH IBM SP2			
500	5,633	10,267	8,195
1000	18,947	28,977	26,216
1500	35,274	49,131	46,093.10
2000	45,186	65,474	62,355

TABLE V
Energy utilisation (J)

NASA Ames iPSC/860			
No of cloudlets	**MOALO**	**Min-Min**	**Max-Min**
500	17,168	28,839	24,249
1000	35,573	54,034	50469
1500	78,628	85,046	79,417
2000	98,066	107,800	99,746
KTH IBM SP2			
500	72,931	143,109	106,968
1000	238,878	385,142	342,254
1500	424,575	642,746	601,757
2000	562,028	858,645	814,054

TABLE VI
Makespan (in Sec)

NASA Ames iPSC/860			
No of cloudlets	**MOALO**	**Min-Min**	**Max-Min**
500	147	182	143
1000	314	336	298
1500	474	507	469
2000	587	659	589
KTH IBM SP2			
500	666	1,032	630
1000	2,127	2,413	2,018
1500	3,611	3,817	3,547
2000	4,940	5,142	4,799

C. Observations and Inferences

The MOALO algorithm was tested on the previously mentioned workloads and compared against other static scheduling algorithms, namely, Max-Min and Min-Min scheduling algorithms. Three objectives were considered and looked into during the testing of the MOALO algorithm, the makespan of the workload, the total power consumption of all datacenters

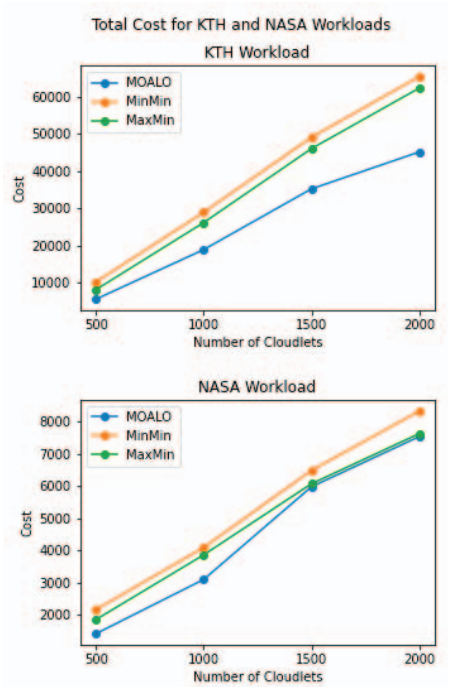

Fig. 3. Cost

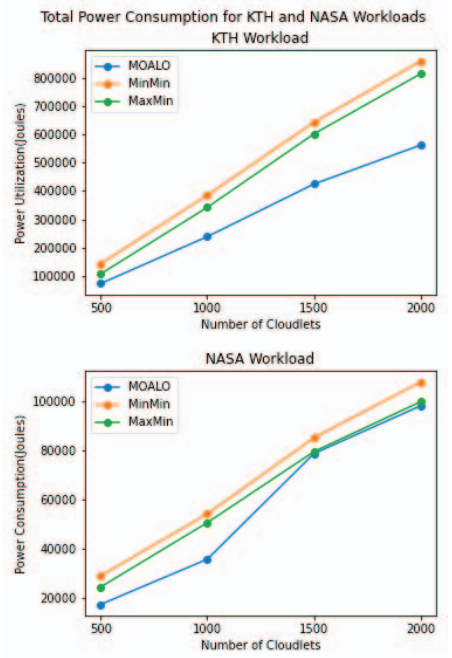

Fig. 4. Power

in the multi-cloud environment and the cost of running VMs while executing cloud tasks. The observed results are depicted

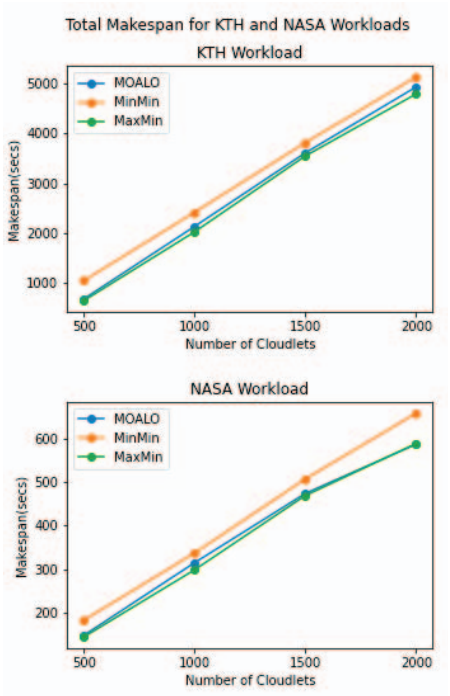

Fig. 5. Makespan

in Table IV, V and VI. Similarly, the metrics are plotted against the number of cloudlets in Fig. 3, Fig. 4, and Fig. 5

Comparing the total costs to run these tasks, it is seen that MOALO performs relatively better than the static algorithms presented. It is almost 16% better than the Max-Min algorithm, about 23% better than Min-Min scheduling algorithm and consistently reduces the overall cost of running resources across both cloud service providers.

Similarly, comparing the total power consumption of these algorithms, it is observed that MOALO performs on an average 20% better than Max-Min and almost 26% better than Min-Min, making it a more power-optimal algorithm compared to the other algorithms presented.

Finally, the makespan of the workload shows a slight drop in performance, as MOALO performs on average 2% worse than Max-Min and around 9% better than Min-Min. It is noticed that MOALO performs relatively well with respect to makespan, as compared to Max-Min.

MOALO succeeds in minimising cost and power utilisation, while maintaining overall makespan, whereas the Max-Min and Min-Min algorithms perform subpar with respect to minimising metrics like cost and power. Since the MOALO algorithm aims to minimise all objective functions, it explores the entire search space to find an optimal allocation. Improvements made using Quasi-oppositional learning and Differential Evolution allow MOALO to optimally allocate initial solutions closest to their respective global minima. On the other hand, Min-Min and Max-Min are greedy algorithms mainly designed

to minimise the completion time of a set of tasks against a set of resources. Thus, such algorithms tend to perform poorly when presented with multi-cloud contextual constraints like cost and power utilisation.

Although MOALO performs relatively better in terms of cost, power utilisation and competitively in terms of makespan, it is noticed that its resulting allocation is skewed towards a cloud service provider with a lower cost per second and power utilisation. This might result in reducing the overall reliability of the system.

VI. Conclusion

Multi-objective cloud task scheduling must not only reduce makespan, but also other objectives like power utilisation and cost. Such an approach is very useful for users adopting the multi-cloud strategy. In this paper, these objectives are minimised using the Multi-Objective Antlion Optimisation (MOALO) algorithm along with Quasi-oppositional learning and Differential Evolution (DE). It can be seen that MOALO optimises cloud resource allocations to minimise the power utilisation of a datacenter and cost for the cloud service user to a significant extent without impacting performance.

The future work consists of improving the MOALO algorithm to achieve lower makespan when compared to the Max-Min algorithm. Other algorithms like NSGA-II and MOPSO can be compared with MOALO in a similar multi-cloud environment to understand how it may perform with respect to the metrics considered in this paper. Further conclusion on MOALO's reliability can be determined in a future study by running the algorithm against additional workloads. Along with this, there is a need to optimise for other constraints in a multi-cloud environment, such as availability, security constraints, CSP level fault tolerance and reliability of the system. Container migrations within and across CSPs can be modelled using the CloudSim framework to further optimise costs and power utilisation against a workload. Finally, the code related to the modelling of a multi-cloud, containerised environment in CloudSim and the training and integration of the MOALO algorithm is to be open-sourced, allowing future researchers to build upon the approaches and algorithms introduced in this work.

References

[1] D. Petcu, "Multi-cloud: expectations and current approaches," pp. 1–6, 04 2013.

[2] P.-C. Quint and N. Kratzke, "Overcome vendor lock-in by integrating already available container technologies - towards transferability in cloud computing for smes," 03 2016.

[3] M. Pinedo and K. Hadavi, "Scheduling: Theory, algorithms and systems development," in *Operations Research Proceedings 1991*, pp. 35–42, Springer Berlin Heidelberg, 1992.

[4] A. Keivani and J.-R. Tapamo, "Task scheduling in cloud computing: A review," in *2019 International Conference on Advances in Big Data, Computing and Data Communication Systems (icABCD)*, pp. 1–6, 2019.

[5] R. Kumar Mondal, E. Nandi, and D. Sarddar, "Load balancing scheduling with shortest load first," *International Journal of Grid and Distributed Computing*, vol. 8, pp. 171–178, 08 2015.

[6] W. Li and H. Shi, "Dynamic load balancing algorithm based on fcfs," in *2009 Fourth International Conference on Innovative Computing, Information and Control (ICICIC)*, pp. 1528–1531, 2009.

[7] H. Chen, F. Wang, N. Helian, and G. Akanmu, "User-priority guided min-min scheduling algorithm for load balancing in cloud computing," in *2013 National Conference on Parallel Computing Technologies (PARCOMPTECH)*, pp. 1–8, 2013.

[8] I. Mahmood, M. M.Sadeeq, S. Zeebaree, H. Shukur, K. Jacksi, H. Yasin, A. Radie, and Z. Najat, "Task scheduling algorithms in cloud computing: A review," *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, vol. 12, pp. 1041–1053, 04 2021.

[9] M. Kumar, S. Sharma, A. Goel, and S. Singh, "A comprehensive survey for scheduling techniques in cloud computing," *Journal of Network and Computer Applications*, vol. 143, pp. 1–33, 2019.

[10] E. H. Houssein, A. G. Gad, Y. M. Wazery, and P. N. Suganthan, "Task scheduling in cloud computing based on meta-heuristics: Review, taxonomy, open challenges, and future trends," *Swarm and Evolutionary Computation*, vol. 62, p. 100841, 2021.

[11] M. Sharma and R. Garg, "An artificial neural network based approach for energy efficient task scheduling in cloud data centers," *Sustainable Computing: Informatics and Systems*, vol. 26, p. 100373, 2020.

[12] F. Ramezani, J. Lu, and F. Hussain, "Task scheduling optimization in cloud computing applying multi-objective particle swarm optimization," pp. 237–251, 12 2013.

[13] F. Ramezani, J. Lu, J. Taheri, and F. Hussain, "Evolutionary algorithm-based multi-objective task scheduling optimization model in cloud environments," *World Wide Web*, vol. 18, 11 2015.

[14] M. A. Elaziz, S. Xiong, K. Jayasena, and L. Li, "Task scheduling in cloud computing based on hybrid moth search algorithm and differential evolution," *Knowledge-Based Systems*, vol. 169, pp. 39–52, 2019.

[15] S. Thennarasu, M. Selvam, and K. Srihari, "A new whale optimizer for workflow scheduling in cloud computing environment," *Journal of Ambient Intelligence and Humanized Computing*, vol. 12, 03 2021.

[16] T. Chaitra, S. Agrawal, J. Jijo, and A. Arya, "Multi-objective optimization for dynamic resource provisioning in a multi-cloud environment using lion optimization algorithm," in *2020 IEEE 20th International Symposium on Computational Intelligence and Informatics (CINTI)*, pp. 000083–000090, 2020.

[17] S. Mirjalili, "The ant lion optimizer," *Advances in Engineering Software*, vol. 83, pp. 80–98, may 2015.

[18] L. Abualigah and A. Diabat, "A novel hybrid antlion optimization algorithm for multi-objective task scheduling problems in cloud computing environments," *Cluster Computing*, vol. 24, 03 2021.

[19] C. Guerrero, I. Lera, and C. Juiz, "Resource optimization of container orchestration: a case study in multi-cloud microservices-based applications," *The Journal of Supercomputing*, vol. 74, 07 2018.

[20] S. Mirjalili, P. Jangir, and S. Saremi, "Multi-objective ant lion optimizer: a multi-objective optimization algorithm for solving engineering problems," *Applied Intelligence*, vol. 46, pp. 79–95, jul 2016.

[21] W. Yadong, S. Quan, S. Mingchang, and S. Weixing, "Quasi-oppositional multi-objective antlion optimizer based on differential evolution," *Journal of Physics: Conference Series*, vol. 1267, p. 012010, july 2019.

[22] S. Rahnamayan, H. Tizhoosh, and M. Salama, "Quasi-oppositional differential evolution," pp. 2229–2236, sept 2007.

[23] R. N. Calheiros, R. Ranjan, C. A. F. De Rose, and R. Buyya, "Cloudsim: A novel framework for modeling and simulation of cloud computing infrastructures and services," 2009.

[24] S. F. Piraghaj, A. V. Dastjerdi, R. N. Calheiros, and R. Buyya, "Containercloudsim: An environment for modeling and simulation of containers in cloud data centers," *Software: Practice and Experience*, vol. 47, no. 4, pp. 505–521, 2017.

[25] B. Kanani and B. Maniyar, "Review on max-min task scheduling algorithm for cloud computing," *Journal of emerging technologies and innovative research*, vol. 2, no. 3, pp. 781–784, 2015.

[26] S. P. E. Corporation, "Hewlett packard enterprise proliant dl360 gen10." https://spec.org/power_ssj2008/results/res2017q3/power_ssj2008-20170704-00768.html, oct 2022.

[27] S. P. E. Corporation, "Lenovo global technology thinksystem sr650." https://spec.org/power_ssj2008/results/res2017q3/power_ssj2008-20170621-00756.html, oct 2022.

[28] "Nasa ames ipsc/860." https://www.cs.huji.ac.il/labs/parallel/workload/l_nasa_ipsc, oct 2022.

[29] "The swedish royal institute of technology (kth) ibm sp2 log." https://www.cs.huji.ac.il/labs/parallel/workload/l_kth_sp2/, oct 2022.